



Figure 1: Illustration of the vanilla model with a segment length 4.

ageable sizes, and only train the model within each segment, ignoring all contextual information from previous segments. This is the idea adopted by Al-Rfou et al. (2018). We call it the *vanilla model* and visualize it in Fig. 1a. Under this training paradigm, information never flows across segments in either the forward or backward pass. There are two critical limitations of using a fixed-length context. First, the largest possible dependency length is upper bounded by the segment length, which is a few hundred on character-level language modeling (Al-Rfou et al., 2018). Therefore, although the self-attention mechanism is less affected by the vanishing gradient problem compared to RNNs, the vanilla model is not able to fully exploit this optimization advantage. Second, though it is possible to use padding to respect the sentence or other semantic boundaries, in practice it has been standard practice to simply chunk long text into fixed-length segments due to improved efficiency (Peters et al., 2018; Devlin et al., 2018; Al-Rfou et al., 2018). However, simply chunking a sequence into fixed-length segments will lead to the context fragmentation problem as discussed in Section 1.

During evaluation, at each step, the vanilla model also consumes a segment of the same length as in training, but only makes one prediction at the last position. Then, at the next step, the segment is shifted to the right by only one position, and the new segment has to be processed all from scratch. As shown in Fig. 1b, this procedure ensures that each prediction utilizes the longest possible context exposed during training, and also relieves context fragmentation issue encountered in training. However, this evaluation procedure is extremely expensive. We will show that our proposed architecture is able to substantially improve the evaluation speed.

3.2 Segment-Level Recurrence with State Reuse

To address the limitations of using a fixed-length context, we propose to introduce a recurrence mechanism to the Transformer architecture. During training, the hidden state sequence computed for the previous segment is *fixed* and *cached* to be reused as an extended context when the model processes the next new segment, as shown in Fig. 2a. Although the gradient still remains within a segment, this additional input allows the network to exploit information in the history, leading to an ability of modeling longer-term dependency and avoiding context fragmentation. Formally, let the two consecutive segments of length L be $\mathbf{s}_\tau = [x_{\tau,1}, \dots, x_{\tau,L}]$ and $\mathbf{s}_{\tau+1} = [x_{\tau+1,1}, \dots, x_{\tau+1,L}]$ respectively. Denoting the n -th layer hidden state sequence produced for the τ -th segment $\mathbf{s}_\tau$ by $\mathbf{h}_\tau^n \in \mathbb{R}^{L \times d}$, where d is the hidden dimension. Then, the n -th layer hidden state for segment $\mathbf{s}_{\tau+1}$ is produced (schematically) as follows,

$$\begin{aligned} \tilde{\mathbf{h}}_{\tau+1}^{n-1} &= [\text{SG}(\mathbf{h}_\tau^{n-1}) \circ \mathbf{h}_{\tau+1}^{n-1}], \\ \mathbf{q}_{\tau+1}^n, \mathbf{k}_{\tau+1}^n, \mathbf{v}_{\tau+1}^n &= \mathbf{h}_{\tau+1}^{n-1} \mathbf{W}_q^\top, \tilde{\mathbf{h}}_{\tau+1}^{n-1} \mathbf{W}_k^\top, \tilde{\mathbf{h}}_{\tau+1}^{n-1} \mathbf{W}_v^\top, \\ \mathbf{h}_{\tau+1}^n &= \text{Transformer-Layer}(\mathbf{q}_{\tau+1}^n, \mathbf{k}_{\tau+1}^n, \mathbf{v}_{\tau+1}^n). \end{aligned}$$

where the function $\text{SG}(\cdot)$ stands for stop-gradient, the notation $[\mathbf{h}_u \circ \mathbf{h}_v]$ indicates the concatenation of two hidden sequences along the length dimension, and $\mathbf{W}$ denotes model parameters. Compared to the standard Transformer, the critical difference lies in that the key $\mathbf{k}_{\tau+1}^n$ and value $\mathbf{v}_{\tau+1}^n$ are conditioned on the extended context $\tilde{\mathbf{h}}_{\tau+1}^{n-1}$ and hence $\mathbf{h}_\tau^{n-1}$ cached from the previous segment. We emphasize this particular design by the green paths in Fig. 2a.

With this recurrence mechanism applied to every two consecutive segments of a corpus, it essentially creates a segment-level recurrence in the hidden states. As a result, the effective context being utilized can go way beyond just two segments. However, notice that the recurrent dependency between $\mathbf{h}_{\tau+1}^n$ and $\mathbf{h}_\tau^{n-1}$ shifts one layer downwards